

Inverse Kinematics

1. Function Description

After the node program starts, it provides a /dofbot_kinematics service. This service provides both fk and ik services. You can choose to call fk or ik by specifying kin.name. The **fk** service takes **the current angle values of the 6 servos (joint values) as input** and **outputs the current end-effector pose of the robotic arm**. This pose includes xyz coordinates and Euler angles (roll, pitch, yaw); The **ik** service takes **the end-effector pose of the robotic arm (xyz coordinates and Euler angle values) as input** and **outputs the angle values of the 6 servos (joints)**. When grasping objects in 3D space later, we need to first read the current end-effector pose of the robotic arm, then identify the center of the object and obtain a new pose coordinate through coordinate transformation, and finally call ik to calculate the value of each servo.

2. Startup and Operation

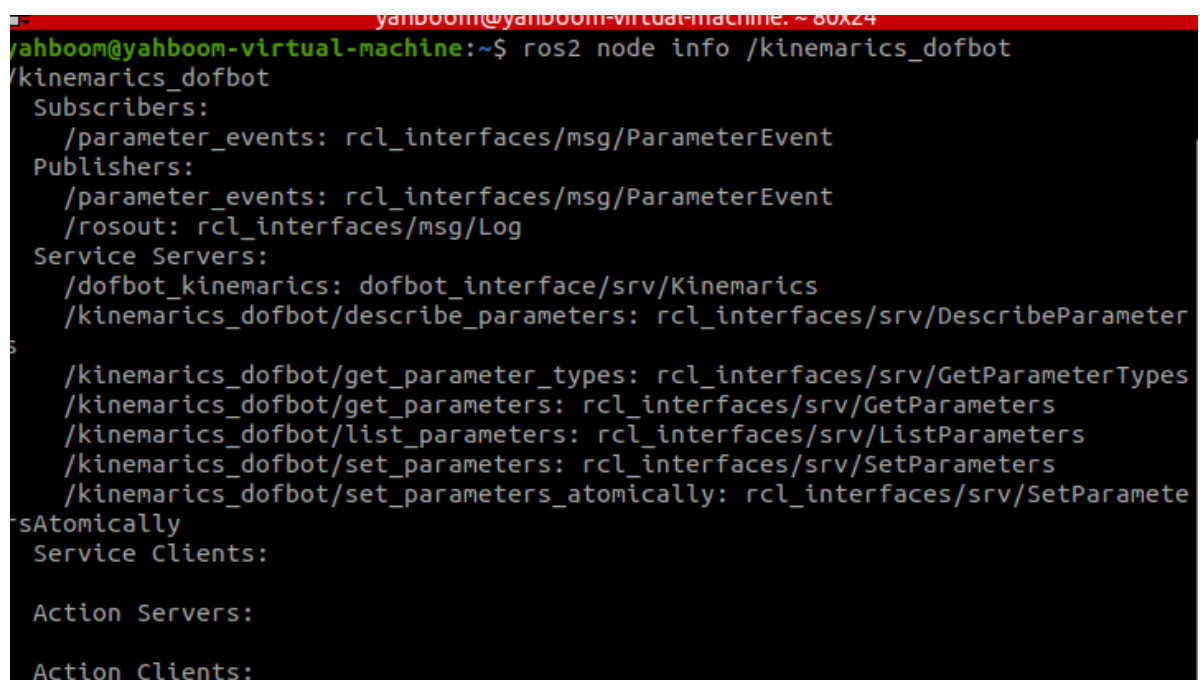
2.1 Startup

Enter in the terminal:

```
# Start the inverse kinematics program
ros2 run dofbot_info kinemarics_dofbot
# Start the ROS control robotic arm program
ros2 run dofbot_driver arm_driver
```

After the inverse kinematics node program starts, it provides a service. To check this, enter in the terminal:

```
ros2 node info /kinemarics_dofbot
```



```
yanboom@yanboom-virtual-machine: ~$ ros2 node info /kinemarics_dofbot
/kinemarics_dofbot
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
Service Servers:
  /dofbot_kinematics: dofbot_interface/srv/Kinemarics
  /kinemarics_dofbot/describe_parameters: rcl_interfaces/srv/DescribeParameter
  /kinemarics_dofbot/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /kinemarics_dofbot/get_parameters: rcl_interfaces/srv/GetParameters
  /kinemarics_dofbot/list_parameters: rcl_interfaces/srv/ListParameters
  /kinemarics_dofbot/set_parameters: rcl_interfaces/srv/SetParameters
  /kinemarics_dofbot/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:
Action Servers:
Action Clients:
```

In the listed service list, there is **/dofbot_kinematics**. This is the inverse kinematics service. Use the following command to view the related information of this service:

```
ros2 service type /dofbot_kinematics
```

```
yahboom@yahboom-virtual-machine:~$ ros2 service type /dofbot_kinematics
dofbot_interface/srv/Kinematics
```

As shown in the image above, the service type data of /dofbot_kinematics is the custom **dofbot_interface/srv/Kinematics**. The specific parameters are the content in Args below. We can use the following command to see what content is included in the **dofbot_interface/srv/Kinematics** data type:

```
ros2 interface show dofbot_interface/srv/Kinematics
```

```
yahboom@yahboom-virtual-machine:~$ ros2 interface show dofbot_interface/srv/Kinematics
# request
float64 tar_x
float64 tar_y
float64 tar_z
float64 roll
float64 pitch
float64 yaw
float64 cur_joint1
float64 cur_joint2
float64 cur_joint3
float64 cur_joint4
float64 cur_joint5
float64 cur_joint6
string kin_name
---
# response
float64 joint1
float64 joint2
float64 joint3
float64 joint4
float64 joint5
float64 joint6
float64 x
float64 y
float64 z
float64 roll
float64 pitch
float64 yaw
```

Here, the --- divides the two parts. The upper part is the request (which can be understood as the input parameters), and the lower part is the response (which can be understood as the output return value). There is an additional kin_name in the request part, which distinguishes whether we are calling IK or FK. If you want to call the IK service, assign IK to kin_name; otherwise, assign the FK value to kin_name to call the FK service. **Parameters in the request part:**

- tar_x: x value of the target position, unit is m
- tar_y: y value of the target position, unit is m
- tar_z: z value of the target position, unit is m
- Roll: Roll value of the target pose, unit is radians
- Pitch: Pitch value of the target pose, unit is radians
- Yaw: Yaw value of the target pose, unit is radians

The above 6 are parameters that must be assigned when calling IK.

- cur_joint1: current angle value of servo #1, unit is degrees
- cur_joint2: current angle value of servo #2, unit is degrees
- cur_joint3: current angle value of servo #3, unit is degrees
- cur_joint4: current angle value of servo #4, unit is degrees
- cur_joint5: current angle value of servo #5, unit is degrees
- cur_joint6: current angle value of servo #6, unit is degrees

The above 6 are parameters that must be assigned when calling FK.

- kin_name: the name of the service to call, ik or fk

Parameters in the response part:

- joint1: angle value of servo #1, unit is degrees
- joint1: angle value of servo #2, unit is degrees
- joint1: angle value of servo #3, unit is degrees
- joint1: angle value of servo #4, unit is degrees
- joint1: angle value of servo #5, unit is degrees
- joint1: angle value of servo #6, unit is degrees

The above 6 values are the 6 servo angle values returned after calling the IK service.

- x: x value of the calculated end-effector position, unit is m
- y: y value of the calculated end-effector position, unit is m
- z: z value of the calculated end-effector position, unit is m
- Roll: Roll value of the calculated end-effector position, unit is radians
- Pitch: Pitch value of the calculated end-effector position, unit is radians
- Yaw: Yaw value of the calculated end-effector position, unit is radians

The above 6 parameters are the end-effector position and pose returned after calling the FK service.

2.2 Operation

2.2.1 FK

First, let's move the robotic arm to a certain state, then based on this state, read out the end-effector position and pose, and compare it with the actual measured values to see if it is accurate. From the previous lesson "ROS Control Robotic Arm", we can use the topic to publish a robotic arm pose first. For example, publish [90,120,30,30,90,90]. Enter in the terminal:

```
ros2 topic pub /TargetAngle dofbot_interface/ArmJoint "id: 0
run_time: 1000
angle: 0.0
joints: [90,120,30,30,90,90]"
```

Press Enter to publish the message, and the robotic arm moves to the specified pose. Then, we call the FK service, writing the just-set [90,120,30,30,90,90] into the current values of the 6 servos, which are cur_joint1 to cur_joint6. Additionally, assign fk to kin_name. Enter in the terminal:

```
ros2 service call /dofbot_kinematics dofbot_interface/srv/Kinematics "{tar_x:
0.0, tar_y: 0.0, tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0,
cur_joint2: 120.0, cur_joint3: 30.0, cur_joint4: 30.0, cur_joint5: 90.0,
cur_joint6: 90.0,
kin_name: 'fk'}"
```

Press Enter, and the response will be printed in the terminal, as shown in the image below:

```
yahboom@yahboom-virtual-machine:~$ ros2 service call /dofbot_kinematics dofbot_interface/srv/Kinematics "{tar_x: 0.0, tar_y: 0.0, tar_z: 0.0, roll: 0.0, pitch: 0.0, yaw: 0.0, cur_joint1: 90.0, cur_joint2: 120.0, cur_joint3: 30.0, cur_joint4: 30.0, cur_joint5: 90.0, cur_joint6: 90.0, kin_name: 'fk'}"
waiting for service to become available...
requester: making request: dofbot_interface.srv.Kinematics_Request(tar_x=0.0, tar_y=0.0, tar_z=0.0, roll=0.0, pitch=0.0, yaw=0.0, cur_joint1=90.0, cur_joint2=120.0, cur_joint3=30.0, cur_joint4=30.0, cur_joint5=90.0, cur_joint6=90.0, kin_name='fk')
response:
dofbot_interface.srv.Kinematics_Response(joint1=0.0, joint2=0.0, joint3=0.0, joint4=0.0, joint5=0.0, joint6=0.0, x=-0.14624000005840623, y=0.0007025520000000001, z=0.26420041152256324, roll=7.3464102067096675e-06, pitch=3.661037138843959e-06, yaw=-3.141592653589793)
```

From the printed response, we can see that the xyz of the robotic arm end-effector position and the Euler angles Roll, Pitch, Yaw representing the robotic arm end-effector pose are: $x=-0.14624000005840623$, $y=0.0007025520000000001$, $z=0.26420041152256324$, Roll= $7.3464102067096675e-06$, Pitch= $3.661037138843959e-06$, Yaw= -3.141592653589793 . From these data, we can know that the robotic arm end-effector is 0.0007025520000000001 meters to the left of the base coordinate system, 0.14624000005840623 meters forward, and 0.26420041152256324 meters high. The pose can be considered unchanged. We can roughly measure the distance from the robotic arm end-effector to the base coordinate system with a ruler to see if it matches the calculated xyz values.

2.2.2 IK

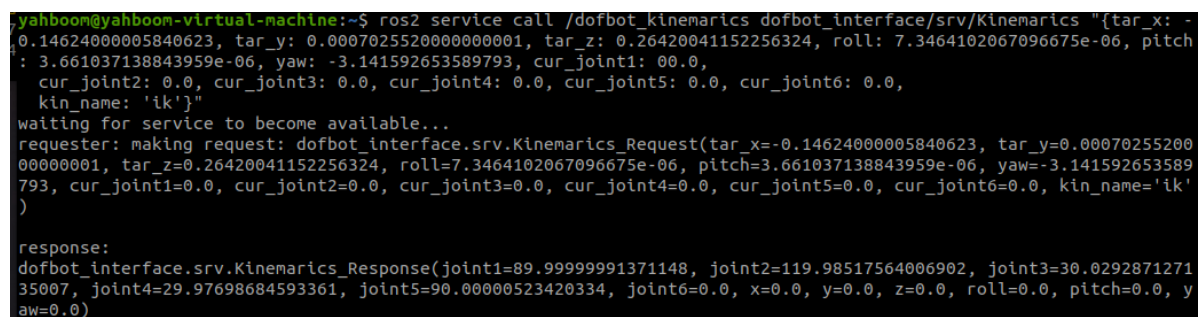
First, let's send the following message to make the robotic arm extend upward. Enter in the terminal:

```
ros2 topic pub /TargetAngle dofbot_interface/ArmJoint "id: 0
run_time: 1000
angle: 0.0
joints: [90,90,90,90,90,90]"
```

Then, we call the IK service. By giving the position and pose parameters of the robotic arm end-effector, we calculate what the values of each servo of the robotic arm should be to achieve this position and pose. Taking the xyz and Euler angles previously calculated by FK as the target position and pose as an example, enter in the terminal:

```
ros2 service call /dofbot_kinematics dofbot_interface/srv/Kinematics "{tar_x: -
-0.14624000005840623, tar_y: 0.0007025520000000001, tar_z: 0.26420041152256324,
roll: 7.3464102067096675e-06, pitch: 3.661037138843959e-06, yaw:
-3.141592653589793, cur_joint1: 0.0,
  cur_joint2: 0.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 0.0,
cur_joint6: 0.0,
  kin_name: 'ik'}"
```

Press Enter, and the response will be printed in the terminal, as shown in the image below:



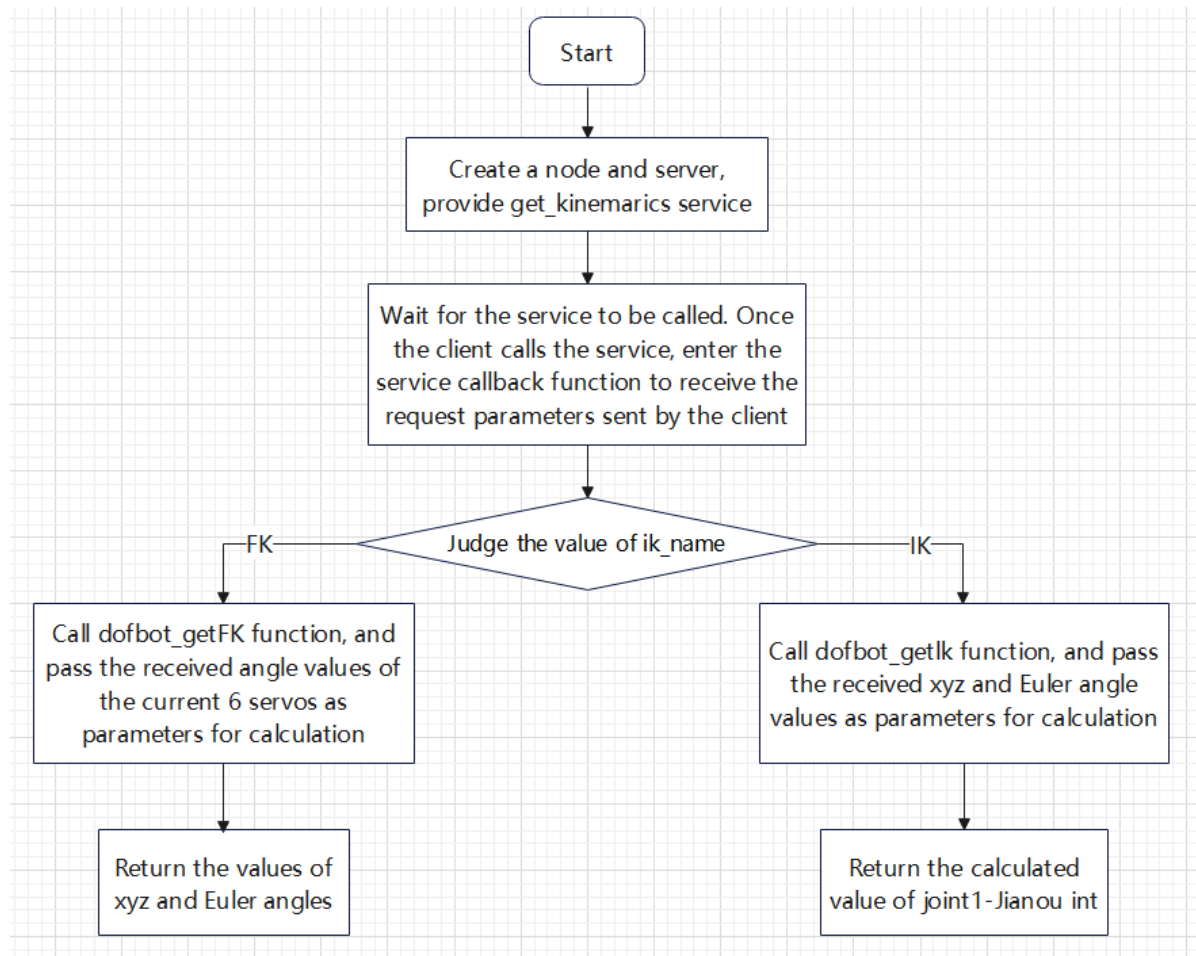
```
yahboom@yahboom-virtual-machine:~$ ros2 service call /dofbot_kinematics dofbot_interface/srv/Kinematics "{tar_x: -
0.14624000005840623, tar_y: 0.0007025520000000001, tar_z: 0.26420041152256324, roll: 7.3464102067096675e-06, pitch
: 3.661037138843959e-06, yaw: -3.141592653589793, cur_joint1: 0.0,
  cur_joint2: 0.0, cur_joint3: 0.0, cur_joint4: 0.0, cur_joint5: 0.0, cur_joint6: 0.0,
  kin_name: 'ik'}"
waiting for service to become available...
requester: making request: dofbot_interface.srv.Kinematics_Request(tar_x=-0.14624000005840623, tar_y=0.00070255200
00000001, tar_z=0.26420041152256324, roll=7.3464102067096675e-06, pitch=3.661037138843959e-06, yaw=-3.141592653589
793, cur_joint1=0.0, cur_joint2=0.0, cur_joint3=0.0, cur_joint4=0.0, cur_joint5=0.0, cur_joint6=0.0, kin_name='ik'
)
response:
dofbot_interface.srv.Kinematics_Response(joint1=89.99999991371148, joint2=119.98517564006902, joint3=30.0292871271
35007, joint4=29.97698684593361, joint5=90.00000523420334, joint6=0.0, x=0.0, y=0.0, z=0.0, roll=0.0, pitch=0.0, y
aw=0.0)
```

From the printed response, we can see that the calculated joint1-joint6 correspond to the values of the 6 servos. After rounding each joint, they are [90,120,30,30,90,0]. Let's try to publish the calculated joint1-joint6 to the robotic arm control node via the topic to make it move to the specified pose. Enter in the terminal:

```
ros2 topic pub /TargetAngle dofbot_interface/ArmJoint "id: 0
run_time: 1000
angle: 0.0
joints: [90,120,30,30,90,0]"
```

Press Enter to make the robotic arm move to the specified pose. Then, we use a ruler to measure whether the distance from the robotic arm end-effector to the base coordinate system matches the xyz we input. A tolerance of 1cm is acceptable. This is because each servo of the robotic arm cannot be guaranteed to be completely identical at the factory, and there are slight differences.

3. Program Flowchart



4. Core Code Analysis

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_info/src/kinematics_dofbot.cpp
```

Import related header files:

```
#include <iostream>
#include <vector>
#include "rclcpp/rclcpp.hpp"
#include <kd1/chain.hpp>
#include <kd1/tree.hpp>
#include <kd1/segment.hpp>
#include <kd1/frames_io.hpp>
#include <kd1/chainiksolverpos_lma.hpp>
#include <kd1/chainfksolverpos_recursive.hpp>
#include <kd1_parser/kd1_parser.hpp>
#include "dofbot_interface/srv/kinematics.hpp"
#include "dofbot_kinematics.h"
```

Define some constants:

```
// Radian to degree
const float RA2DE = 180.0f / M_PI;
// Degree to radian
const float DE2RA = M_PI / 180.0f;
// URDF model, when doing inverse kinematics, the urdf model will be used as a
parameter input for calculation
const char *urdf_file =
"/home/yahboom/dofbot_ws/src/dofbot_urdf/urdf/dofbot.urdf";
```

Main function:

```
auto server_node = rclcpp::Node::make_shared("kinemarics_dofbot");
auto service = server_node->create_service<dofbot_interface::srv::Kinemarics>(
    "dofbot_kinemarics", srvicecallback);
```

Service callback function srvicecallback:

FK service part:

```
if (request->kin_name == "fk") {
    // Get the current joint angles from the service request
    // 从服务请求中获取当前关节角度
    double joints[] {
        request->cur_joint1, request->cur_joint2, request->cur_joint3,
        request->cur_joint4, request->cur_joint5
    };

    // Container for the initial joint angles
    // 初始关节角度容器
    vector<double> initjoints;

    // Container for the resulting end-effector position and orientation
    // 末端执行器位置和姿态的结果
    vector<double> initpos;

    // Convert joint angles from degrees to radians
    // 将关节角度从度转换为弧度
    for (int i = 0; i < 5; ++i) {
        initjoints.push_back((joints[i] - 90) * DE2RA);
    }

    // Call the forward kinematics function to calculate end-effector pose
    // 调用正向运动学函数计算末端执行器位姿
    if (getFK(urdf_file, initjoints, initpos)) {
        // Log the FK input joint angles for debugging
        // 记录FK输入关节角度以便调试
        RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
            "FK Input Joints: J1=%.2f, J2=%.2f, J3=%.2f, J4=%.2f,
J5=%.2f",
            joints[0], joints[1], joints[2], joints[3], joints[4]);

        // Log the FK output pose (position + orientation)
        // 记录FK输出位姿（位置 + 姿态）
```

```

        RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
                    "FK Output Pose: X=%.4f, Y=%.4f, Z=%.4f, Roll=%.4f,
Pitch=%.4f, Yaw=%.4f",
                    initpos.at(0), initpos.at(1), initpos.at(2),
                    initpos.at(3), initpos.at(4), initpos.at(5));

        // Fill the response with the computed pose
        // 将计算得到的位姿填入响应
        response->x = initpos.at(0);
        response->y = initpos.at(1);
        response->z = initpos.at(2);
        response->roll = initpos.at(3);
        response->pitch = initpos.at(4);
        response->yaw = initpos.at(5);
    }
    .....
}

```

IK service part:

```

if (request->kin_name == "ik") {
    // Define the end-effector pose (position and orientation)
    // 定义末端执行器位姿（位置和姿态）
    double xyz[]{request->tar_x, request->tar_y, request->tar_z};
    double rpy[]{request->roll, request->pitch, request->yaw};

    // Vectors to store target position, orientation, and output joint angles
    // 存储目标位置、姿态和输出关节角的向量
    vector<double> targetXYZ;
    vector<double> targetRPY;
    vector<double> outjoints;

    // Copy input position values to targetXYZ vector
    // 将输入的位置值复制到targetXYZ向量
    for (int k = 0; k < 3; ++k) targetXYZ.push_back(xyz[k]);

    // Copy input orientation values (RPY) to targetRPY vector
    // 将输入的姿态值（RPY）复制到targetRPY向量
    for (int l = 0; l < 3; ++l) targetRPY.push_back(rpy[l]);

    // Log the IK input parameters for debugging
    // 记录IK输入参数以便调试
    RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
                "IK Input Target: X=%.4f, Y=%.4f, Z=%.4f, Roll=%.4f, Pitch=%.4f,
Yaw=%.4f",
                xyz[0], xyz[1], xyz[2], rpy[0], rpy[1], rpy[2]);

    // Call the inverse kinematics solver
    // 调用逆运动学求解器
    if (getIK(urdf_file, targetXYZ, targetRPY, outjoints)) {
        // Log the IK output joint angles
        // 记录IK输出的关节角度
        RCLCPP_INFO(rclcpp::get_logger("rclcpp"),
                    "IK Output Joints: J1=%.2f, J2=%.2f, J3=%.2f, J4=%.2f,
J5=%.2f",
                    (outjoints.at(0) * RA2DE) + 90,
                    (outjoints.at(1) * RA2DE) + 90,

```

```
        (outjoints.at(2) * RA2DE) + 90,  
        (outjoints.at(3) * RA2DE) + 90,  
        (outjoints.at(4) * RA2DE) + 90);  
  
    // Set the response joint angles  
    // 设置响应关节角度  
    response->joint1 = (outjoints.at(0) * RA2DE) + 90;  
    response->joint2 = (outjoints.at(1) * RA2DE) + 90;  
    response->joint3 = (outjoints.at(2) * RA2DE) + 90;  
    response->joint4 = (outjoints.at(3) * RA2DE) + 90;  
    response->joint5 = (outjoints.at(4) * RA2DE) + 90;  
}  
    .....  
}
```